

Plano de Implementação — Cache de EPUB com IndexedDB + Memória

Objetivo: Eliminar o re-download do arquivo EPUB a cada visita ao leitor, tornando a abertura instantânea após o primeiro acesso e permitindo leitura offline.

Cenário Atual

Usuário clica em "Ler agora"

|



getBookById(id) + getUserBooks() ← ~200ms

|



fetch(/api/epub?url=...) ← ~2s (download do EPUB inteiro)

|



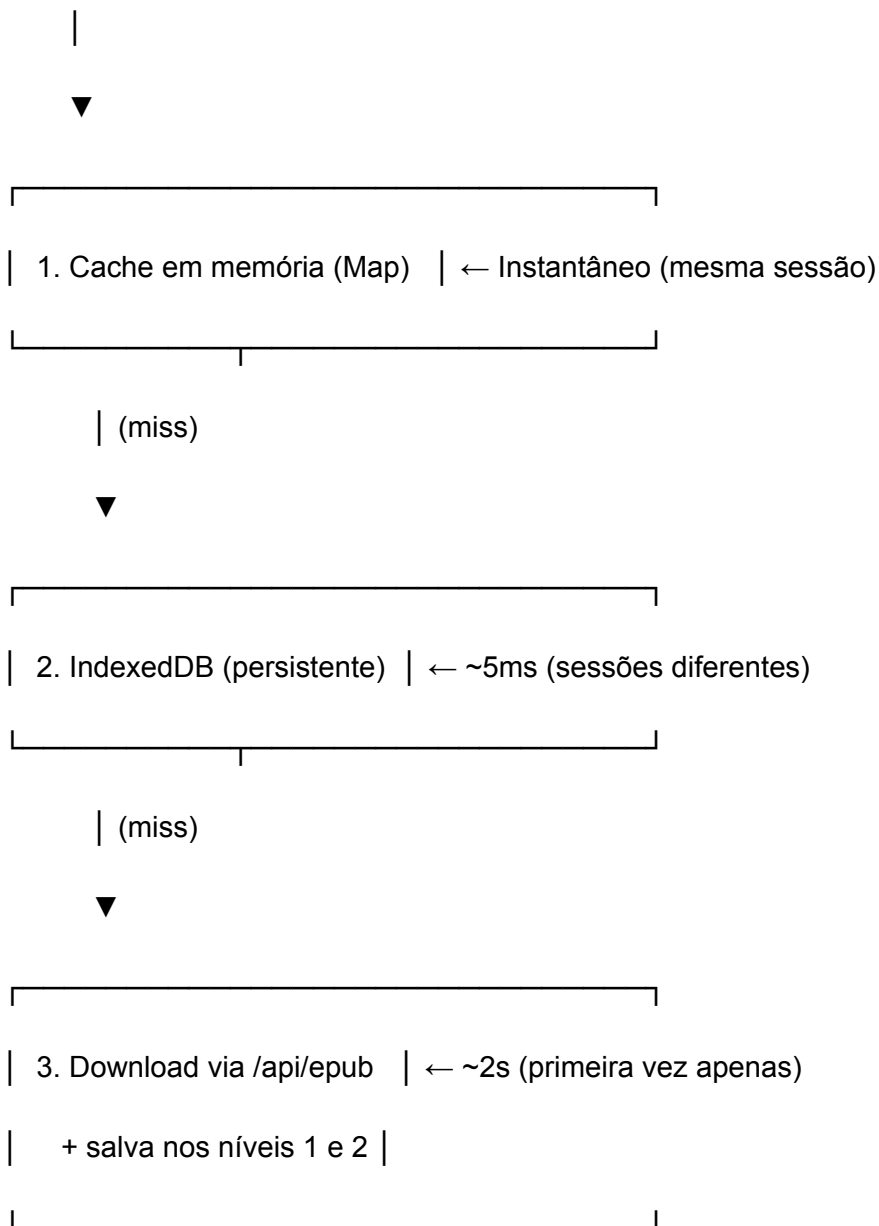
setEpubData(arrayBuffer) ← renderiza o ReactReader

Problema: Toda vez que o componente `<LeitorPage>` é montado, o EPUB é baixado do zero — mesmo que o usuário já tenha lido aquele livro antes.

Solução Proposta

Estratégia de cache em 3 níveis

Solicitação do EPUB



Arquivos a Criar

1. `alexandria-frontend/src/lib/epub-cache.ts` (NOVO)

Funções para gerenciar o cache em memória e IndexedDB.

// Cache em memória (Map) — dura enquanto a aba estiver aberta

```
const memoryCache = new Map<string, ArrayBuffer>();

// Constantes do IndexedDB

const DB_NAME = "alexandria-epub-cache";

const STORE_NAME = "epubs";

const DB_VERSION = 1;

/**

* Abre (ou cria) o banco IndexedDB.

* A store é chave-valor: bookId → ArrayBuffer

*/

function openDB(): Promise<IDBDatabase> {

  return new Promise((resolve, reject) => {

    const request = indexedDB.open(DB_NAME, DB_VERSION);

    request.onupgradeneeded = () => {

      // Cria a object store se não existir

      request.result.createObjectStore(STORE_NAME);

    };

    request.onsuccess = () => resolve(request.result);

    request.onerror = () => reject(request.error);

  });

}
```

* Tenta obter o EPUB de um livro.

*

* @returns ArrayBuffer com o conteúdo do EPUB, ou null se não encontrado

*/

```
export async function getCacheEpub(bookId: string): Promise<ArrayBuffer | null> {
```

```
  // 1. Tenta cache em memória (instantâneo)
```

```
  if (memoryCache.has(bookId)) {
```

```
    console.debug(`[epub-cache] HIT (memory): ${bookId}`);
```

```
    return memoryCache.get(bookId)!.slice(0);
```

```
  }
```

```
  // 2. Tenta IndexedDB (persistente entre sessões)
```

```
  try {
```

```
    const db = await openDB();
```

```
    const data = await new Promise<ArrayBuffer | undefined>((resolve, reject) => {
```

```
      const transaction = db.transaction(STORE_NAME, "readonly");
```

```
      const request = transaction.objectStore(STORE_NAME).get(bookId);
```

```
      request.onsuccess = () => resolve(request.result);
```

```
      request.onerror = () => reject(request.error);
```

```
    });
```

```
    db.close();
```

```
    if (data) {
```

```
      console.debug(`[epub-cache] HIT (indexeddb): ${bookId}`);
```

```

    // Promove para memória

    memoryCache.set(bookId, data);

    return data.slice(0);

}

} catch (error) {

    console.warn("[epub-cache] Erro ao ler IndexedDB:", error);

}

// 3. Cache miss

console.debug(`[epub-cache] MISS: ${bookId}`);

return null;

}

/**

* Salva um ArrayBuffer nos dois níveis de cache (memória + IndexedDB).

*/

export async function saveEpubToCache(bookId: string, data: ArrayBuffer): Promise<void> {

    // Salva na memória

    memoryCache.set(bookId, data);

    // Salva no IndexedDB (assíncrono, não bloqueia)

    try {

        const db = await openDB();

        const transaction = db.transaction(STORE_NAME, "readwrite");

```

```

transaction.objectStore(STORE_NAME).put(data, bookId);

await new Promise<void>((resolve, reject) => {

    transaction.oncomplete = () => resolve();

    transaction.onerror = () => reject(transaction.error);

});

db.close();

console.debug(`[epub-cache] Saved to IndexedDB: ${bookId}`);

} catch (error) {

    console.warn("[epub-cache] Erro ao salvar no IndexedDB:", error);

}

}

/**

* Remove um EPUB do cache (útil se o usuário quiser liberar espaço).

*/

export async function removeEpubFromCache(bookId: string): Promise<void> {

    memoryCache.delete(bookId);

    try {

        const db = await openDB();

        const transaction = db.transaction(STORE_NAME, "readwrite");

        transaction.objectStore(STORE_NAME).delete(bookId);

        await new Promise<void>((resolve, reject) => {

            transaction.oncomplete = () => resolve();

```

```

    transaction.onerror = () => reject(transaction.error);

  });

  db.close();

} catch (error) {

  console.warn("[epub-cache] Erro ao remover do IndexedDB:", error);

}

}

```

2. Modificar

[alexandria-frontend/src/app/\(main\)/leitor/\[id\]/page.tsx](#)

O que muda:

```

import { getBookById, getUserBooks, updateUserBook, getAuthorDisplay } from "@lib/api";
+ import { getCacheEpub, saveEpubToCache } from "@lib/epub-cache";

useEffect(() => {

  async function init() {

    const [b, userBooks] = await Promise.all([

      getBookById(Number(id)),

      getUserBooks().catch(() => []),

    ]);

    setBook(b);

    // ... processa userBook ...

-   if (b.downloadUrl) {

-     const res = await fetch(`/api/epub?url=${encodeURIComponent(b.downloadUrl)}`);

```

```
-   if (res.ok) setEpubData(await res.arrayBuffer());  
  
- }  
  
+ // Tenta carregar do cache primeiro  
  
+ if (b.downloadUrl) {  
  
+   const cached = await getCachedEpub(id);  
  
+   if (cached) {  
  
+     // Cache hit — instantâneo  
  
+     setEpubData(cached);  
  
+   } else {  
  
+     // Cache miss — baixa da API e salva no cache  
  
+     try {  
  
+       const res = await fetch(`/api/epub?url=${encodeURIComponent(b.downloadUrl)}`);  
  
+       if (res.ok) {  
  
+         const data = await res.arrayBuffer();  
  
+         setEpubData(data);  
  
+         // Não espera o cache salvar para não atrasar a UI  
  
+         saveEpubToCache(id, data);  
  
+       }  
  
+     } catch (err) {  
  
+       console.error("Falha ao baixar EPUB:", err);  
  
+     }  
  
+   }  
  
+ }  
  
+ }
```



```

+   }

+   }

  }

  init().catch(console.error).finally(() => setLoading(false));

}, [id]);

```

Arquivo final esperado — parte do **useEffect**:

```

useEffect(() => {

  async function init() {

    const [b, userBooks] = await Promise.all([

      getBookById(Number(id)),

      getUserBooks().catch(() => []),

    ]);

    setBook(b);

    const userBook = userBooks.find((ub) => ub.book.id === Number(id));

    if (userBook) {

      userBookIdRef.current = userBook.id;

      const savedProgress = userBook.progress ?? 0;

      lastSavedProgressRef.current = savedProgress;

      currentProgressRef.current = savedProgress;

      setProgress(savedProgress);

      if (userBook.status === "toread") {

        updateUserBook(userBook.id, { status: "reading", progress: 0 }).catch(() => {});

```

```

    }
}

const saved = localStorage.getItem(`epub-location-${id}`);

if (saved) restoreLocationRef.current = saved;

// Tenta cache primeiro, depois baixa

if (b.downloadUrl) {

    const cached = await getCacheEpub(id);

    if (cached) {

        setEpubData(cached);

    } else {

        try {

            const res = await fetch(`/api/epub?url=${encodeURIComponent(b.downloadUrl)}`);

            if (res.ok) {

                const data = await res.arrayBuffer();

                setEpubData(data);

                saveEpubToCache(id, data);

            }

        } catch (err) {

            console.error("Falha ao baixar EPUB:", err);

        }

    }

}

```

```
}  
  
}  
  
init().catch(console.error).finally(() => setLoading(false));  
  
// ... cleanup ...  
  
}, [id]);
```

Fluxo Completo (Depois da Implementação)

Primeira vez que abre o livro

Usuário clica em "Ler agora"

|



getBookById(id) + getUserBooks() ← ~200ms

|



getCachedEpub(id) ← ~5ms → MISS

|



fetch(/api/epub?url=...) ← ~2s (download)

|



setEpubData(data) ← renderiza o ReactReader

saveEpubToCache(id, data) ← salva em memória + IndexedDB (paralelo)

|



✓ Leitor pronto

Segunda vez (mesma sessão)

Usuário clica em "Ler agora"

|



getBookById(id) + getUserBooks() ← ~200ms

|



getCachedEpub(id) ← instantâneo → HIT (memória)

|



setEpubData(data) ← renderiza o ReactReader

|



✓ Leitor pronto — sem download

Terceira vez (sessão diferente, mesmo livro)

Usuário clica em "Ler agora"

|



getBookById(id) + getUserBooks() ← ~200ms

|



getCachedEpub(id) ← ~5ms → HIT (IndexedDB)

|

(promove para memória)



setEpubData(data) ← renderiza o ReactReader

|



✓ Leitor pronto — sem download

Considerações

Tamanho do cache

- EPUB médio: 500KB a 5MB
- IndexedDB suporta centenas de MB sem problemas
- O usuário pode liberar espaço manualmente limpando dados do navegador

Tratamento de erros

- Se IndexedDB falhar (disco cheio, modo anônimo restritivo), o cache em memória ainda funciona para a sessão atual
- Se IndexedDB e memória falharem, faz o download normal (graceful degradation)

Cache HTTP da rota `/api/epub`

Já existe `Cache-Control: public, max-age=86400` — complementa o IndexedDB. Se o usuário nunca abriu o livro antes, o proxy `/api/epub` retorna do cache HTTP do navegador se já tiver sido baixado em outra aba.

Limpeza do cache (futuro)

Pode-se adicionar uma opção em "Configurações" para limpar o cache de EPUBs:

```
import { removeEpubFromCache } from "@lib/epub-cache";

async function clearEpubCache() {

  const keys = Object.keys(localStorage).filter(k => k.startsWith("epub-location-"));

  for (const key of keys) {

    const bookId = key.replace("epub-location-", "");

    await removeEpubFromCache(bookId);

  }

}
```

Checklist de Implementação

- ☐ Criar `alexandria-frontend/src/lib/epub-cache.ts` com funções:
 - `openDB()` — abrir/criar IndexedDB
 - `getCachedEpub(bookId)` — busca cache (memória → IndexedDB)
 - `saveEpubToCache(bookId, data)` — salva nos dois níveis
 - `removeEpubFromCache(bookId)` — remove do cache
- ☐ Modificar `leitor/[id]/page.tsx`:
 - Importar `getCachedEpub` e `saveEpubToCache`
 - Substituir `fetch` direto por: cache → download → salvar
- ☐ Testar:
 1. Abrir livro → esperar download → leitor carrega 
 2. Fechar e reabrir o mesmo livro → instantâneo 
 3. Fechar a aba, abrir nova aba → instantâneo (IndexedDB) 
 4. Modo anônimo/privado → funciona (cai para download se IndexedDB não estiver disponível) 